

第九单元：BOM

1. 简介

1.1. BOM 是什么？

1.2. BOM 组成部分

2. window 对象

2.1. 常见方法

3. location 对象

3.1. 主要属性

3.2. 主要方法

3.3. 示例

4. history 对象

4.1. 概述

4.2. 主要属性和方法

5. screen 对象

5.1. 主要属性

6. navigator 对象

6.1. 主要属性

7. 总结

8. 案例

1. 简介

1.1. BOM 是什么？

- BOM 全称是 **浏览器对象模型**（Browser Object Model），浏览器实现的供 JavaScript 程序调用的一组对象

1.2. BOM 组成部分

1. **window** 对象

- **功能**：表示浏览器的一个窗口或框架，是BOM的核心对象，包含了文档对象（DOM）。

2. **location** 对象

- **功能**：提供有关当前URL的信息，并允许JavaScript修改浏览器的URL。

3. **history** 对象

- **功能**：当前浏览器的历史记录信息，包含用户在浏览器窗口中访问过的URL。

4. **navigator** 对象

- **功能**：包含关于浏览器自身的信息，如浏览器的名称、版本等。

5. **screen** 对象

- **功能**：提供关于用户屏幕的相关信息。

6. **document** 对象

- **功能**：当前页面文档相关信息，可以操作页面上的元素。

2. window 对象

所有在全局声明的变量和函数，自动成为 window 对象的属性和方法。

2.1. 常见方法

注意：无需显式使用 window 前缀调用全局变量和全局方法

1. alert()

- **功能**：显示带有一段消息和一个确认按钮的警告框。
- **语法**：window.alert(message)
- **示例**：

```
1 window.alert("这是一个警告框！");  
2 alert("这是一个警告框！");
```

2. confirm()

- **功能**：显示带有一段消息以及确认和取消按钮的对话框。
- **语法**：window.confirm(message)

3. prompt()

- 功能：显示带有一段消息以及一个文本框和确认/取消按钮的对话框。
- 语法：window.prompt(text, defaultText)

4. setTimeout()

- 功能：延时器，在指定的毫秒数后调用函数。
- 语法：window.setTimeout(function, milliseconds)
- 示例：

```
1 ▼ setTimeout(() => {  
2     console.log("3秒后执行");  
3 }, 3000);
```

5. clearTimeout()

- 功能：取消由 setTimeout() 设置的延时器。
- 语法：window.clearTimeout(timeoutID)
- 示例：

```
1 ▼ var timeoutID = setTimeout(function () {  
2     /* ... */  
3 }, 3000);  
4 clearTimeout(timeoutID);
```

6. setInterval()

- 功能：定时器，重复调用一个函数，每隔一定时间间隔执行一次。
- 语法：window.setInterval(function, milliseconds)
- 示例：

```
1 ▼ var intervalID = setInterval(() => {  
2     console.log("每3秒执行一次");  
3 }, 3000);
```

7. clearInterval()

- 功能：取消由 `setInterval()` 设置的定时器。
- 语法：`window.clearInterval(intervalID)`
- 示例：

```
JavaScript |
1 var intervalID = setInterval(() => {
2     console.log("每3秒执行一次");
3 }, 3000);
4 clearInterval(intervalID);
```

8. open()

- 功能：打开新窗口或查找已命名的窗口。
- 语法：`window.open(URL, name)`
- 示例：

```
JavaScript |
1 window.open("https://www.example.com", "_blank");
```

9. close()

- 功能：关闭当前窗口。
- 语法：`window.close()`
- 示例：

```
JavaScript |
1 window.close();
```

3. location 对象

- 作用：获取当前页面 url 地址信息

3.1. 主要属性

1. href：完整的URL。
2. protocol：URL的协议部分，如'http:'、'https:'。
3. host：服务器名称和端口号（如果有）。

4. **hostname**: 服务器名称。
5. **port**: URL的端口号。
6. **pathname**: URL的路径部分。
7. **search**: URL的查询字符串部分, 包括开头的?。
8. **hash**: URL的锚部分, 包括开头的#。

3.2. 主要方法

1. **assign(url)**: 跳转页面 (有历史记录) 。
2. **reload(forceGet)**: 重新加载当前页面。
3. **replace(url)**: 用新的URL替换当前的页面(不会产生历史记录)。
4. **href** 可以获取当前url完整地址, href = “值” 可以设置url地址
5. **search** 获取当前地址的用户信息部分(查询信息), 返回? 及后面的信息
6. **hash**获取#锚的部分

3.3. 示例

JavaScript

```
1 console.log(location.href); // 当前页面的完整URL
2 console.log(location.protocol); // 如 'http:'
3 console.log(location.host); // 如 'www.example.com:80'
4 console.log(location.hostname); // 如 'www.example.com'
5 console.log(location.port); // 如 '80'
6 console.log(location.pathname); // 如 '/path/index.html'
7 console.log(location.search); // 如 '?q=query'
8 console.log(location.hash); // 如 '#section'
9
10 // 跳转到新页面
11 location.href = "https://www.example.com";
12 location.assign("https://www.example.com");
13
14 // 重新加载页面
15 location.reload();
16
17 // 替换当前页面
18 location.replace("https://www.anotherexample.com");
```

4. history 对象

4.1. 概述

- **作用**：主要用于在用户的浏览历史中前进和后退，以及对历史记录堆栈进行操作。

4.2. 主要属性和方法

1. length

- **功能**：返回当前会话中历史列表中的URL数量。

2. back()

- **功能**：返回上一页，相当于浏览器的“后退”按钮。

3. forward()

- **功能**：跳转下一个URL，相当于浏览器的“前进”按钮。

4. go(delta)

- **描述**：按照历史记录中的相对位置加载页面，**delta**可以是正数（前进）或负数（后退）。
- **示例**：

JavaScript

```
1 history.go(-1); // 后退一页
2 history.go(0);  // 刷新页面
3 history.go(1);  // 前进一页
```

5. pushState(state , title , url)

- **描述**：在不重新加载页面的情况下修改浏览器的url地址,并将新的url添加到浏览器的历史记录中。
- **参数**:
 - **state** 状态对象是一个由pushState()方法创建的，与历史记录相关的javascript对象，当用户定向到一个新的状态时，会触发popstate事件；事件的state属性包含了历史记录的状态对象；如果不需要该对象可写null。
 - **title** 新页面的标题【注意：所有的浏览器目前都忽略这个值，因此可写为null】
 - **url** 新历史记录的地址【注意: 新url必须和当前url在同一个域，否则pushState方法抛出异常】

- **注意:** `pushState()`方法不会触发页面刷新，只是导致`history`对象发生变化，地址栏的显示地址发生变化。
- **示例:**

```
▼ pushState () JavaScript |
1  history.pushState({ a : 1 , b : 2 } , 'pushState' , './demo.html')
2  // 假设当前进入页面的url地址是index.html,由于修改了url地址为demo.html
3  // 即 页面内容不会加载demo的信息，只是地址栏显示demo.html
4
5  举例:
6  history.pushState( { page : 1 } , 'title 1' , '?page1' )
7  history.pushState( { page : 2 } , 'title 2' , '?page2' )
8  history.replaceState( { page : 3 } , 'title 3' , '?page3' )
9  history.back()
10 // url显示为http://localhost:52331/0725/history.html?page1
11
12 history.back()
13 // url显示为http://localhost:52331/0725/history.html
14
15 history.go(2)
16 // url显示为http://localhost:52331/0725/history.html?page3
```

6、state属性

描述: 是浏览器历史记录中当前页面的状态对象，通过 `history.pushState()` 或 `history.replaceState()` 方法创建或修改。该对象通常包含与当前页面相关的状态信息，例如用户数据或页面配置

- **状态管理:** 在单页应用（SPA）中，`history.state` 可用于存储页面状态信息（如用户输入、滚动位置等），无需重新加载页面即可恢复状态。 ³⁴
- **事件触发:** 用户通过浏览器后退/前进按钮或 `history.go()` 方法切换页面时，若使用 `pushState/replaceState` 修改历史记录，会触发 `popstate` 事件并携带当前状态对象
- **示例:**

```
▼ JavaScript |
1  console.log( history.state )
2  // { page : 3 }
```

5. screen 对象

- **作用：**主要用于获取有关用户显示屏幕的大小和颜色能力等信息。

5.1. 主要属性

1. **width：**屏幕的宽度（以像素为单位）。
2. **height：**屏幕的高度（以像素为单位）。
3. **availWidth：**屏幕的可用宽度（以像素为单位），不包括界面特性，如任务栏。
4. **availHeight：**屏幕的可用高度（以像素为单位），不包括界面特性，如任务栏。

6. navigator 对象

- **作用：**主要用于获取浏览器相关信息，如名称、版本、所启用的插件等。

6.1. 主要属性

1. **appName:** 浏览器的官方名称。
2. **appVersion:** 浏览器的版本信息。
3. **userAgent:** 浏览器发送到服务器的用户代理头的值
4. **platform:** 浏览器运行的操作系统平台。
5. **language:** 浏览器当前设置的语言。
6. **onLine:** 表示浏览器是否处于在线状态。
7. **geolocation:** 提供设备的地理位置信息。

7. 总结

1. BOM 的组成部分
2. **setTimeout 和 setInterval 使用和区别**
3. location 对象的常用属性和方法
4. history 对象的常用方法

8. 案例

1. 打字机效果

开始打字

你站在桥上看风景,看风景的人在桥下看你,明月装饰了你的窗,你装饰了别人的梦

2. 验证码倒计时

6秒后重新获取

3. 日期倒计时

04天12时34分1秒

4. 点名表 (案例地址)

谢国军	赵聪明	刘洋	沙如超	唐文鑫
王青云	李靖鹏	杨佳宇	钱昱康	徐敬太
夏启明	田凯旋	徐玮	邓汉	董也川
夏正堃	陈旭	王昕睿	王越	陈胜勇
沈天乐	顾银贵	潘镀	许崇林	

开始点名